Year Up United
Data Analyst Training Academy: SQL Data Fundamentals

Week 2 Lab Workbook

This courseware is to be distributed to accompany instructor led training.

Copyright © 2026 Pluralsight & The Reliable Collaboration Company

1.A.1 Table of Contents

MODULE 1 INTRODUCTION TO RELATIONAL DATABASES AND SQL 3

EXERCISE 1.A GETTING STARTED WITH MYSQL WORKBENCH 4
EXERCISE 1.B SCRATCHING THE SURFACE WITH SAKILA 5
EXERCISE 1.C UNDERSTANDING THE SAKILA DATABASE DESIGN 9

MODULE 2 RELATIONAL DATABASE DESIGN 11

EXERCISE 2.A NORMALIZATION PRACTICE 12
EXERCISE 2.B GROUP WORKSHOP: DESIGNING A DATABASE 14

MODULE 3 WORKING WITH TABLES 18

EXERCISE 3.A WORKSHOP PART 2: IMPLEMENTING THE DATABASE 19

MODULE 4 WORKING WITH RECORDS 22

EXERCISE 4.A WORKSHOP PART 3INSERTING RECORDS INTO TABLES 23
EXERCISE 4.B IMPORTING AND EXPORTING DATA 25
EXERCISE 4.C ADDING THE NORTHWIND DATABASE 26

Module 1

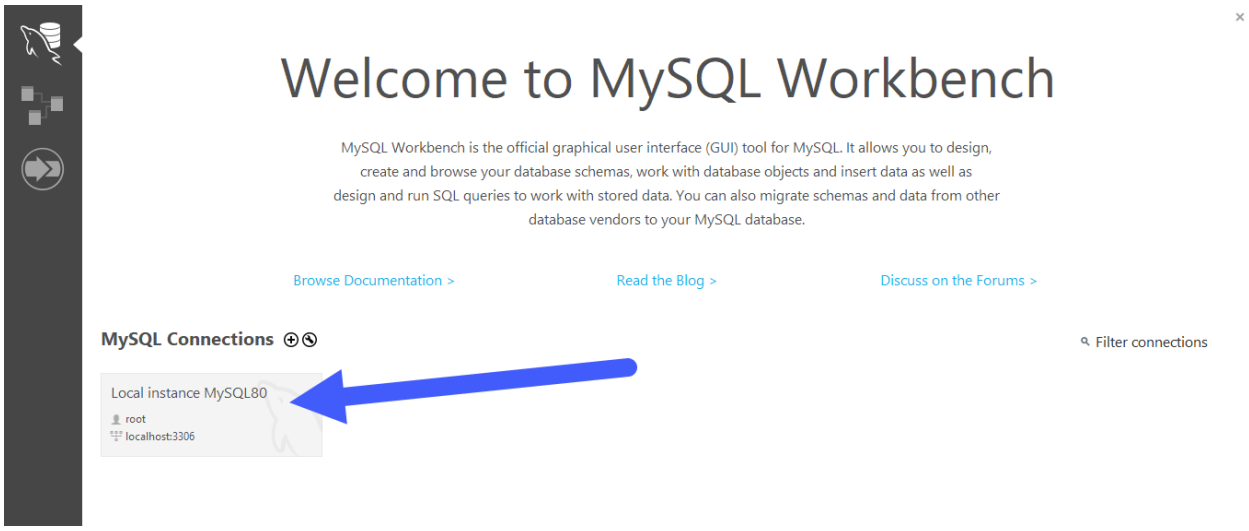
Introduction to Relational Databases and SQL

Exercise 1.A Getting Started with MySQL Workbench

Getting Ready

This week you will begin to write SQL queries using MySQL Workbench. You will continue sharing your saved homework via GitHub.

1. Open MySQL Workbench and ensure that you have a "Local instance" set up under MySQL Connections.



   a) If you do not have MySQL Workbench installed, you will need to work through the instructions to install it on your laptop, or work with your instructor for help troubleshooting.

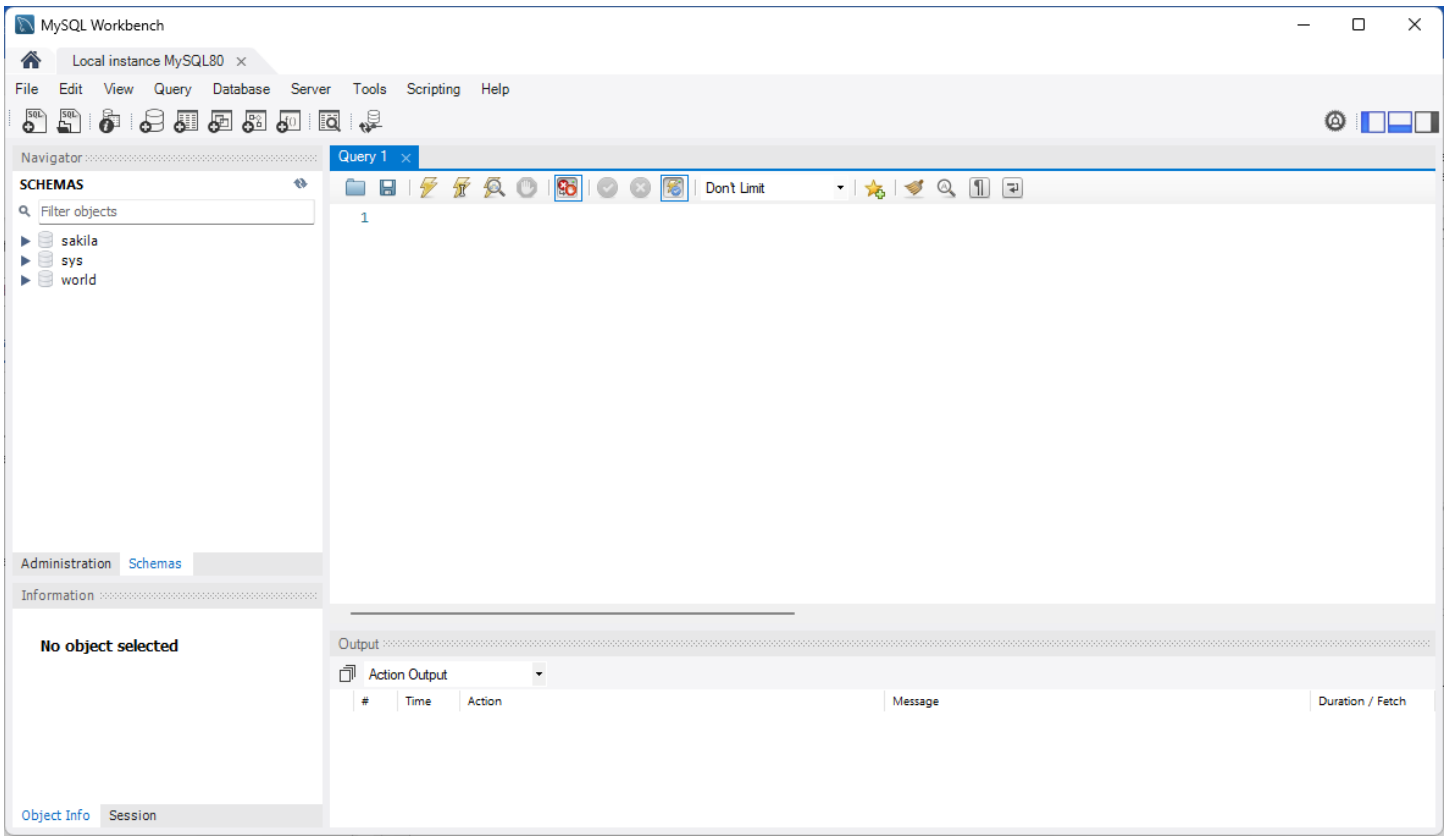
   b) If you are able to open MySQL Workbench, but do not see anything appearing under MySQL Connections, use the plus icon to setup a new connection with the name "Local instance". If that does not work or if you run into problems, work with your instructor for help troubleshooting.

Exercise 1.B Scratching the surface with sakila

Exploring the "sakila" data set

1. Open MySQL Workbench and connect to your "local instance" server.

2. If this is the first time you are opening MySQL Workbench, your view will default to a blank query, with available schemas listed in the navigator panel at the left.



3. Under SCHEMAS on the left, double-click on **sakila**.

4. Double-click on Tables to expand the list of tables in this database. Spend a few minutes studying the names of the tables and consider what this database might be used for.

5. Click in the blank query window under the tab for Query 1. Write a multi-line comment as follows:

a) On the first line, type `/*` then hit enter. Then on line 2, type what you think might be the purpose of the sakila database, and hit enter. Finally on line 3, type `*/` to close.

b) The symbols /* and */ indicate that the text in between them is a comment, meaning that it is text provided for reference, not active code.

c) Your full comment should look something like this:

```
1  /*
2  Sakila is a database used to collect information about _____ and _____ for use by _____.
3  */
```

6. Save this query using File > Save Script As (or Ctrl+Shift+S). Save it to the folder DataAnalytics/week-02 with the name **Ex1B_first_query.sql**

7. As you work through this lab exercise, begin building the habit of not only saving your work frequently, but also regularly committing those changes to GitHub.

a) If you do not already have a terminal open (e.g. Git Bash, PowerShell, Command Prompt), open a new terminal window. Use the `cd` command to change directory to DataAnalytics and then week-02.

b) In DataAnalytics/week-02, use the command `git status` to view what changes have happened that affect this repository. You should get something similar to this:

```
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        Ex1B_first_query.sql

nothing added to commit but untracked files present (use "git add" to track)
```

c) Add all file changes to commit by typing the following command, then hit enter:
`git add .`

d) Create a commit and include a commit message with the following command (you can modify the text inside the quotes if needed for your specific update). Then hit enter:
`git commit -m "added comment to file"`

e) Optional: You can check whether your commit was successful by entering the command `git status`. This should return the message "Your branch is ahead

of ‘origin/main’ by 1 commit” and close with “nothing to commit, working tree clean”

f) Finally, push your changes to the remote server by typing the following command, then hitting enter:
```
git push
```

g) After you push your changes, there should be a return message starting with “Enumerating objects…” and concluding with a summary of changes. If you don’t get any error messages, then your changes were successfully published. To confirm that your branch is up to date and the working tree is clean, use the command `git status`.

8. Head back to MySQL Workbench.

9. In your query Ex1B_first_query, leave a blank line after the comment. On the next line, enter the following two SQL statements, then hit the lightning bolt icon in the query toolbar (⚡)
```
USE sakila;
SELECT * FROM actor;
```

a) Note that since you double-clicked sakila from the Navigator panel earlier, it should already be the active database, and you don’t strictly need the `USE` statement.

b) However, when you save your script, you want to make sure that it includes appropriate instructions on what database to use for the next person who opens and runs that script (in the workplace that might be a colleague, while in this class it’s likely to be your instructor).

10. A list of records should appear. In the output window at the bottom of the screen, you should also see a success message that tells you how many rows of data were returned.

11. To the right of your actor query, add two hyphens and the comment “Retrieved [number] records” to make a note of how many records were retrieved. The two hyphens indicate an inline comment. Your updated query should look like this:
```
SELECT * FROM actor; -- Retrieved [number] records
```

12. Hit enter to add a blank line and again for a new line. Type a new SELECT query, this time on the film table, then hit the icon for the lightning bolt with the cursor over it ( ):
    ```
    SELECT * FROM film;
    ```

    The icon runs only the specific query statement under the keyboard cursor. This can be useful when, as here, you're working with multiple SELECT statements (or other types of queries).

13. Check how many records were retrieved with the query of the film table, and add an inline comment, similar to step 11:
    ```
    SELECT * FROM film; -- Retrieved [number] records
    ```

14. Save changes to the query file using File > Save Script (or Ctrl+S).

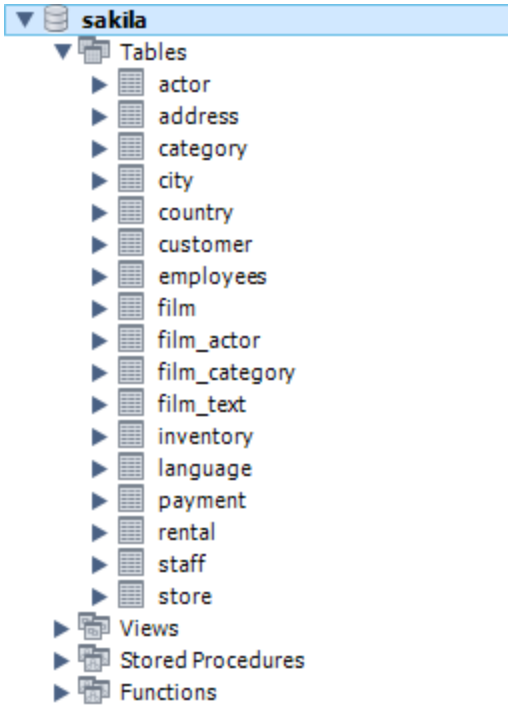
15. In the terminal window, in DataAnalytics/week-02, use the following commands to add, commit, and push your changes to the remote server:
    ```
    git add .
    git commit -m "Added queries of actor and film tables"
    git push
    ```

16. When finished, go to your repository on github.com and check out what your updates look like online. Where can you see the commit messages you added?

Exercise 1.C Understanding the sakila database design

Reviewing sakila tables and relationships

1. In MySQL Workbench, expand the **`sakila`** schema to see all tables.



2. Open a new blank query and save it to week-02 as **Ex1C_sakila_exploration.sql**

3. On the first line in the query editor, start a multi-line comment by typing `/*` then hit enter. Type a) plus a space, and hit enter again. Finally on line 3, type `*/` to close the comment. The final result should look like this:

```
/*
a)
*/
```

Use the inside of this multi-line comment to answer the below questions in step 4, continuing to add new lines for each letter (b, c, etc.) like this:

```
/*
a) Answer to first question as a full sentence.
b) Answer to second question as another full sentence.
(etc. until the end of step 4) */
```

4. Answer the following questions inside the multiline comment you set up above.
   a) In the schema panel, expand the table for **`actor`** and then expand Columns. What information is included in this table?
   b) Now expand the table for **`film`** and then expand Columns. What information is included in this table?
   c) Which other table in sakila contains columns for both **`actor_id`** and **`film_id`**?
   d) Right-click on **`rental`** and choose the menu option for Select Rows. Notice that this opens a new query window. Look at the query results to determine what information you can find in the **`rental`** table. What does this table include? Is this information easy to read, or hard to read? Why?
   e) Perform the same steps for the inventory table. What does this table include?
   f) Based on what you have learned so far, which tables do you need to use in order to understand the names of all films that were rented on a specific date? How are these tables related to each other? Answer with an explanation in full sentences.

5. Underneath the multiline comment, leave a blank line. Then write a SELECT statement for each table that you identified in your answer to (f) above, retrieving just the relevant columns that will help answer the question ("what were the names of all films that were rented on a specific date?"). Each SELECT statement for each specific table can go on a new line, like this:

```
SELECT ___________ FROM _______;
SELECT ___________ FROM _______;
```
(etc.)

6. Save your changes to **Ex1C_sakila_exploration.sql** using File > Save Script (or Ctrl+S).

7. Open Git Bash and navigate to DataAnalytics/week-02, then use the following commands to add, commit, and push your changes to the remote server:
```
git add .
git commit -m "Exercise 1.C complete"
git push
```

Module 2

Relational Database Design

Exercise 2.A Normalization Practice

Normalizing Data into Multiple Tables

This exercise is intended to help you wrap your head around data normalization concepts. You will use Microsoft Excel for this exercise to make it a little easier to lay everything out visually.

1. From the course material, download the starting file **events_sample.xlsx** then open it using Excel.

2. Using the knowledge you have developed so far of how relational databases are structured and the "normal forms", break the sample data down into *at least* three related tables, using the space provided in the spreadsheet. Consider:

3. Which columns can be grouped together to describe a particular thing in the data? What will you call that table? Put the name of each table in a yellow highlighted box.

4. Copy and paste the records from the original data into the appropriate new table. Be sure to eliminate any duplicate records based on the new table structure.

5. In your new tables, highlight the primary key column for each table in blue.

6. When you are finished, save changes to your week-02 folder as **Ex2A_events_normalized.xlsx** then close the Excel file.

7. Go to your preferred AI chat tool, e.g. Claude or ChatGPT, and start a new chat.

8. Upload the spreadsheet to the chat and add a prompt like this:

   At the top of the attached spreadsheet is a dump of flattened data. I have broken it out into tables using normalization rules. Can you please review this spreadsheet and give me some feedback?

   Then hit submit.

9. Review the AI-generated feedback. Do not assume it is 100% right! Some feedback may be helpful and on-point, and some may miss the mark.

10. Reopen the spreadsheet **Ex2A_events_normalized.xlsx** and make any changes you think are appropriate after reviewing the AI feedback. At the bottom of your spreadsheet, underneath the tables, add any additional comments that you have related to the AI feedback (e.g. what was helpful versus what was not helpful).

11. Save changes to the spreadsheet, and make sure you have it saved to your week-02 folder. Then use Git Bash to add, commit, and push your file changes.

Exercise 2.B Group Workshop: Designing a Database

With your study group:

Imagine you have a friend named Lana who has started her own dog-walking business, and she is asking you for help in figuring out what data she will need to keep track of.

Coordinate your group collaboration using asynchronous chat and by scheduling group meetings for active discussion. Group meetings should take place during your lab time – do your best to coordinate around any other individual schedule constraints.

Share repository setup

Designate someone in your group to create a new GitHub repository for this project, named **W2_Workshop**, including a README file that lists all group members.

Meanwhile, on your local computer, in your Documents folder create a new directory for DATA_Labs and navigate to that directory in Git Bash, then clone the W2_Workshop repository into that folder. To do all three of these steps in Git Bash, use the following commands:

```
mkdir DATA_Labs
cd DATA_Labs/
git clone [URL for your group's W2_Workshop repository].git
```

Example:

```
bilbo@computer MINGW64 ~/Documents
$ mkdir DATA_Labs

bilbo@computer MINGW64 ~/Documents
$ cd DATA_Labs/

bilbo@computer MINGW64 ~/Documents/DATA_Labs
$ git clone https://github.com/[illegible]/W2_Workshop.git
Cloning into 'W2_Workshop'...
remote: Enumerating objects: 6, done.
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (6/6), done.

bilbo@computer MINGW64 ~/Documents/DATA_Labs
$ |
```

As your group members separately push their changes to the repository over the course of the next few exercises, you will need to incorporate a new step into your Git Bash update workflow: Always use `git status` first to check for remote changes, then use `git pull` to pull down those changes into your local repository.

A new Git command: **`git pull`**

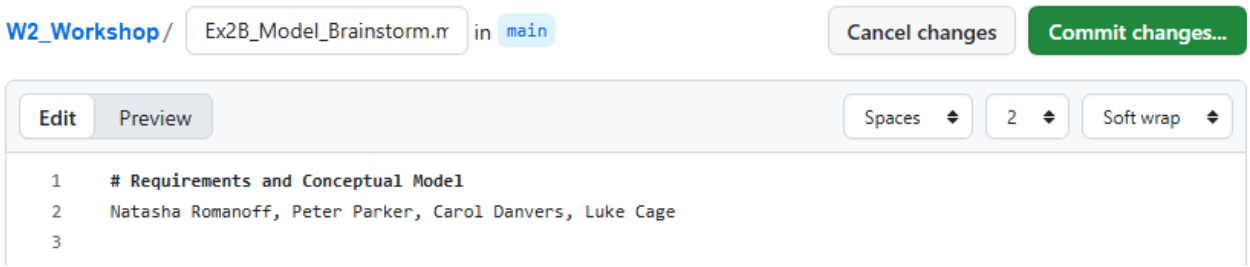
So far you have used `git push` to send your local commits up to the remote repository. When you are the only person working in a repository, that's all you need. But now that your whole group is pushing changes to the same shared repository, your local copy can fall behind if someone else has pushed new work that you don't have yet.

The solution: Before pushing your own changes, always check whether the remote repo has updates you need to pull down first, using the following three commands:

```
git status
git pull
git push
```

This sequence checks for and fetches any new commits from the remote repository and merges them into your local copy, *before* pushing your changes. If you skip this step and try to push while the remote is ahead of you, Git will reject your push and ask you to pull first anyway. Making it a habit saves you that extra step.

1. As a group, pick someone to create a new file in GitHub in W2_Workshop named **Ex2B_Model_Brainstorm.md** and use it to take notes.
   a) Add a header at the top, # **Requirements and Conceptual Model.** (The hash symbol will format the line as a header in the markdown file.) Beneath that, add the names of all study group members.



   b) Brainstorm some basic requirements for this project. Who is the “end user” and what will the end user need to be able to do? What different categories of data might be important to track?

c) For each broad category of data that you come up with, what specific pieces of information would be useful to track? For each category, create a list of bullets with what information that category includes.

d) Make sure that the person who created the document saves changes and pushes their update to the W2_Workshop remote repository, and that other members of the group then each update their local repository using the command: **`git pull`**

2. Continue editing with **Ex2B_Model_Brainstorm.md** (either in GitHub or locally by opening the file in Notepad) and add a new header, #**Logical Model**. Under the new header, continue making notes:

a) What should be the main entities in your database (which will translate into tables)? Aim for at least three, but not more than five, to keep things from getting overly complicated.

b) What attributes (column names) belong to each of those entities? It may be helpful to consider an example of a single record belonging to that entity – what are the pieces of information that the record should include?

c) What entities will have relationships to each other? That is, what information will need to be reassembled together when data is retrieved from the database?

d) Can you identify any possible many-to-many relationships between your tables that will need to a resolving table?

e) Save changes to the notes document, push to the remote repository, and ensure all group members update their local repository to pull down those changes.

On your own:

3. Each group member should try this separately on their own:

a) Go to your preferred AI chat tool and start a new chat. Create a prompt to describe the entities and attributes that your group has defined for your model. (Again, aim for at least three, not more than five.) Also include your notes on expected relationships between tables. Ask for a draft entity relationship diagram (ERD) along with any feedback on the table structure.

b) Make revisions as needed, and continue prompting until you get an ERD that you are happy with. Also take this opportunity to ask clarifying questions about what different parts of the diagram mean and how it would be used in practice.

c) Save your final diagram to W2_Workshop using the filename **lastname_Ex2B_ERD** (either by downloading the diagram image file or by taking a screenshot then saving it).

d) In Git Bash, remember to use ***git status*** to check for other repository changes (and if yes, ***git pull***) before you push your own saved changes.

With your group:

4. Reconvene with your small group and take turns reviewing each of your diagrams. Have one person open **Ex2B_Model_Brainstorm.md** and take notes on your group's conversation as you discuss the questions below.

   a) What are the similarities between individual diagrams? What turned out the same?

   b) Where did each of your diagrams differ? What explanation do you have for those differences? What might be the pros and cons of each approach?

5. Save changes to the notes document, push to the remote repository, and ensure all group members update their local repository to pull down those changes.

Module 3

Working with Tables

Exercise 3.A Workshop Part 2: Implementing the Database

Lana's Dog-Walking Business, Continued

Continue to save all project documentation to W2_Workshop. The first part of 3.A below is independent work, but feel free to share questions, tips, and other collaborative help with your group members throughout the project.

On your own:

1. Open a new Excel workbook. Save to W2_Workshop as **lastname_Ex3A_tables.xlsx**
   a) Note: Why are you being asked to include your last name in the filename? Consider that you are saving this homework file to a shared repository that multiple classmates are contributing to. So, each homework file needs a distinct name, and ideally that name should identify the person who contributed it!

2. Using entity relationship diagram and other notes you created for the last exercise:
   a) Use the first sheet of your workbook to create the first table for Lana's dog-walking database. Rename the sheet to use the same name as your table.
   b) In the top row (row 1), enter the column names that will be used for this table.
   c) Add a new sheet to your workbook for each table you want to create, putting the appropriate column names at the top of each sheet.
   d) For each table, identify the column you expect to be the primary key, and highlight it in light blue.
   e) For each table, identify any foreign key columns, and highlight those in light green

3. Open MySQL Workbench and create a new query. Save it to W2_Workshop as **lastname_Ex3A_create_tables.sql**
   a) Start your script with the following SQL statement:
   ```
   DROP SCHEMA IF EXISTS lana_dog_walking;
   CREATE SCHEMA lana_dog_walking;
   USE lana_dog_walking;
   ```

Run the script and refresh schemas to confirm that the new schema was created. Then save changes (Ctrl+S). Use Git Bash to check first for any new changes to W2_Workshop that you need to pull, then push your changes to W2_Workshop.

4. Go to your preferred AI chat tool and start a new chat.

5. One at a time, write a prompt to ask for help creating a MySQL script to create each table that you defined in your Excel workbook above. Name the table you want to create and list its column names. Indicate which column is the expected primary key and which (if any) are foreign keys. Also note what data type each column is expected to use (string, integer, decimal number, date or datetime, Boolean, etc.)

   Example:
   I'm creating a database for a dog-walking business. Create a MySQL script to add a table named Pets with columns for Pet Name (string), Pet Type (string), Last Walk (datetime), Next Walk (datetime), and Customer Number (integer).
   (This is just an example! Your tables might be a bit different.)

   a) As you work through generating a script to create each table, pay attention to the notes returned by the chat and adjust as needed.

   b) For instance, you may receive feedback about appropriate primary key and foreign key columns – incorporate that feedback and refresh the script. You may also see notes about whether a given table is dependent on another table already being in existence – pay attention to these notes!

   c) However, you may disregard any notes about creating an index, triggers, stored procedures, or other enhanced functionality. Focus on creating tables.

6. Back in MySQL Workbench, add each CREATE TABLE script to your file **lastname_Ex3A_create_tables.sql**

   a) You will need to add tables in the appropriate dependency order. That is, if a table uses a foreign key, the other table it references must exist first, so that is the first in dependency order.

   b) Try running your script as you incorporate new tables to see if the addition is successful or if any error message is generated. Use those error messages as feedback to help figure out what to try next. (If it's unclear what an error message means, try asking your AI chat tool for an explanation.)

c) Note that if you run the complete script, it will always start by dropping the old version and recreating a new version. That's why the script starts with a DROP command, to create a clean slate.

7. Be sure to save your work as you go! When you are satisfied that your script is working to add all your tables to the lana_dog_walking schema, save changes and push to the W2_Workshop repository. (Remember to check for changes to pull first.)

With your group:

8. Reconvene with your small group and take turns reviewing each of your table creation scripts in the W2_Workshop repository.

   a) Have everyone take a turn to share their screen and walk through the script they created. Are there differences across your group in how you each implemented tables? What drove the different choices that were made?

   b) Because all of your scripts are in the shared repository, you can also easily open and run a script created by one of your classmates on your own machine. Note that if you do so, the same DROP/CREATE commands at the top will cause their version of the schema to replace yours. Run your own script again to restore your version.

Module 4

Working with Records

Exercise 4.A Workshop Part 3Inserting Records into Tables

Lana's Dog-Walking Business, Continued

Continue to save all project documentation to W2_Workshop. The first part of 3.A below is independent work, but feel free to share questions, tips, and other collaborative help with your group members throughout the project.

On your own:

1. Open MySQL Workbench and create a new query. Save it to W2_Workshop as **lastname_Ex4A_insert_records.sql**
   a) Start your script with the following SQL statement:
   `USE lana_dog_walking;`

2. Open the Excel workbook with your tables and columns, **lastname_Ex3A_tables.xlsx**
   a) Review the workbook. Make any necessary updates based on changes you made to your tables while you worked through your create_tables script, such as adding, updating, or deleting columns, or adding new sheets for any additional tables. When you are done updating, your table names and column names should exactly match the names used in your final SQL script.
   b) On the sheet for each table, under the column names, create a first row of sample data by populating an invented value for each column. Be sure to save your changes.

3. Go to your preferred AI chat tool and start a new chat.
   a) Start with the first table in your Excel workbook. In the AI chat, write a prompt that provides the exact table name and column names for the table, along with the first row of sample data you came up with. Ask the chat to return an additional 10 sample records based on the provided format, with an `INSERT` script to insert the records into your table.
   b) Pay attention to the notes returned by the chat and make adjustments as needed. For instance, your insert script may not need to include the primary key column where that value is going to be generated by the database. If it helps, you can

provide the create_tables script to the chat for clarification of how each table is set up.

4. When it looks like you have a working draft of an insert script with sample records, go to MySQL Workbench and add the insert statement to your open script, **lastname_Ex4A_insert_records.sql**
   a) Run the insert script for the table. Then run a SELECT query for the table to see how the records look (e.g. `SELECT * FROM _____;`)
   b) If your records don't look as expected, try querying the AI chat tool for troubleshooting help. You may want to delete and re-add the table to try again. To delete a table, use the statement `DROP TABLE ____;` with the name of the table in the blank. Then go to your create_tables script, highlight just the table you want to re-add, and run just that portion of the script.
   c) Save changes to your insert_records script with each set of records you add.

5. Use Git Bash to check first for any new changes to W2_Workshop that you need to pull, then push your changes to W2_Workshop.

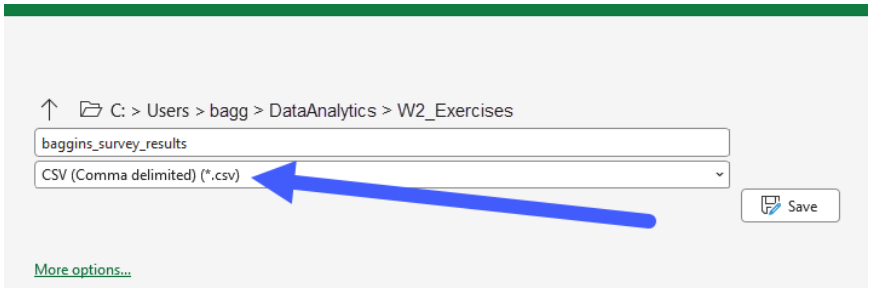
With your group:

6. Reconvene with your small group and take turns reviewing each of your insert scripts in the W2_Workshop repository.
   a) How was your experience with using AI chat to help create sample data? Was it a useful troubleshooting tool? What mistakes did it make?
   b) How does your sample data look? What things can you start to determine about this business, based on the sample records you've created?
   c) Everyone in your small group started from the same brainstorming document. After working independently on implementation, how different have your databases turned out? What are the most notable differences?

Exercise 4.B Importing and exporting data

Importing Survey Data

1. Remember the survey you created last week? Download the results as an Excel file and open the file in Excel. Then use **Save As** to save a copy to week-02 with the filename **Ex4B_survey_results.csv**, using the dropdown options to convert to a .csv file type.



2. In MySQL Workbench, create a new schema named after the restaurant from your survey. Use the Table Data Import wizard to import your survey results as a table in your database.

   a) If you encounter any data-related errors that prevent you from importing the survey results, open the .csv file and make any cleanup changes necessary to remove the error.

3. Write a `SELECT` statement to review the imported results in Workbench.

4. Then use the Table Data Export wizard to export the result set as a .json file, and save it to your week-02 folder with the name **Ex4B_survey_results.json**

Exercise 4.C Adding the Northwind Database

Adding a Database from a Script

1. Add the Northwind database using the following steps:
   a) Use the Notepad application on your computer to open **Northwind_MySQL.sql** from the Week 2 student material.
   b) Copy the full text of the script, and **paste into a blank query** in MySQL Workbench.
   c) Run the query to create the Northwind database.

2. Refresh the Schemas panel to confirm that you have a new schema for **`northwind`**.

3. Create a script named **Ex4C_northwind_extract.sql** and save it to your week-02 folder.

4. Answer the following questions by studying the northwind schema. At the top of your script, include comments with your answers (written as a complete sentence):
   a) What is the name of the table that holds the items Northwind sells?
   b) What is the name of the table that holds the types/categories of the items Northwind sells?

5. Create a `SELECT` statement to retrieve all columns from the **employees** table.
   a) Who is the Northwind employee whose name makes it look like she's a bird? Include the answer as a comment underneath the `SELECT` statement.

6. Create a `SELECT` statement to retrieve all columns from the **products** table.
   a) How many records does your query return? Using the toolbar options at the top of the query pane, how can you change your query to retrieve only 10 rows? Include the answer as a comment underneath the `SELECT` statement.
   b) **BONUS**: How could you write the query to limit the number of rows returned? This isn't covered in the Week 2 Student Guide, so if you tackle this bonus question, you'll need to do a little independent research. Add the answer as a comment in your script with a note on what source you used to find the answer.

7. Create another `SELECT` statement to retrieve all columns from the **categories** table.

   c) What is the category id of seafood? Include a comment in your script to answer this.

8. Create a final `SELECT` statement to retrieve the top 50 records from **orders**, including only the OrderID, OrderDate, ShipName, and ShipCountry columns.

   a) Export the resulting record set to .csv format as **Ex4C_order_sample.csv** and save it to week-02.

9. Save changes to **Ex4C_northwind_extract.sql**

10. Using Git Bash, commit and push your changes to week-02 to GitHub.

End of Week 2 Workbook